Smart Café – Product Requirement Document

1. Problem and Value

The aim of Smart Café is to make ordering food and drinks easier for customers. Instead of waiting in a queue, customers can use the app to browse the menu, select products and place an order for collection. Customers will also be able to track their order and receive notifications when it is ready.

2. User Roles

The main user of the application will be the customer. Customers will be able to register, log in, browse the menu, add products to their cart, place orders, track their progress and leave feedback.

As I am working individually, I will only be developing the Customer application. The admin side will be simulated through automatic order updates to show how the café would process orders.

3. Project Scope

The application will include customer registration and login, a café menu, product searching and filtering, a shopping cart, order placement, order tracking, notifications, order history and customer feedback.

The menu will include hot drinks, cold drinks, cakes, sandwiches and snacks. Customers will be able to place Click & Collect orders and pay when they arrive at the café. They will also be able to cancel orders before preparation begins.

The application will not include online payments, delivery services, product customisation or a full Admin system.

4. Functional Requirements

FR01 – Customers can register using their full name, email address and password, with password confirmation.

FR02 – Customers can securely log in and log out of their accounts.

FR03 – Customers can browse the café menu and view product names, descriptions, prices and availability.

FR04 – Customers can search for products and filter the menu using category tabs.

FR05 – Customers can add products to their cart, change quantities and remove products.

FR06 – Customers can place Click & Collect orders and receive confirmation.

FR07 – Customers can cancel orders while their status is Pending.

FR08 – Customers can track orders as their statuses automatically update.

FR09 – Customers receive app and Android phone notifications about their orders.

FR10 – Customers can view their previous orders.

FR11 – Customers can rate completed orders from 1–5 stars and leave an optional comment.

5. Nonfunctional Requirements

Usability – The application should be easy to use, with simple navigation and a clear layout.

Security – Customer information must be protected, and users should only have access to their own account and orders.

Performance – The application should respond quickly when browsing the menu and placing orders.

Reliability – The application should handle incorrect inputs and connection errors without crashing.

Maintainability – The code should be organised so that it is easy to understand and make changes when needed.

Data Accuracy – Product prices, shopping cart totals and order details should be calculated and stored correctly.

6. Data Requirements

The application will need to store customer details, menu products, shopping cart information, orders and feedback.

Customer information will include a customer ID, full name and email address. Product information will include a product ID, name, category, description, price and availability. Orders will include an order ID, customer ID, selected products, quantities, total price, date and status. Feedback will include an order ID, rating and optional comment.

Customer passwords will be managed through Firebase Authentication rather than being stored directly in the database. Order and feedback information will be stored using Firebase.

7. Technical Constraints

The application will be developed using Kotlin in Android Studio, as required by the coursework. I plan to use Jetpack Compose for the user interface, Firebase Authentication for customer accounts and Cloud Firestore for storing application data.

GitHub will be used throughout development to save my code and track changes.

For the advanced technical integration, I plan to use Firebase for cloud based order tracking. When an order is placed, its status will automatically change from Pending to Preparing after 15 seconds, then to Ready for Collection after another 30 seconds, and finally to Collected after another 30 seconds.

These updates will be stored in Firebase and displayed to the customer, along with notifications. The collection stage will be simulated rather than confirming that the customer has physically collected the order. This is my initial idea, after feedback and the more I progress with this job this feature might change to something else.

8. Acceptance Criteria

The application will be considered functional when:

1. Customers can register and log in successfully using valid details, while incorrect information displays an error.
2. Customers can log out and no longer access protected account information.
3. The menu displays the correct products, prices, descriptions and availability.
4. Customers can search and filter products.
5. Customers can add, remove and update products in their cart, with the total price calculated correctly.
6. Customers can place orders and receive confirmation.
7. Customers can cancel orders while pending but not once preparation has started.
8. Order statuses update automatically, and customers receive notifications.
9. Customers can view their order history and leave ratings and feedback.

Review and Changes

This is the first version of my PRD. I will review it with my lab tutor and make any necessary changes based on their feedback. Any changes to the requirements will be recorded throughout development.